

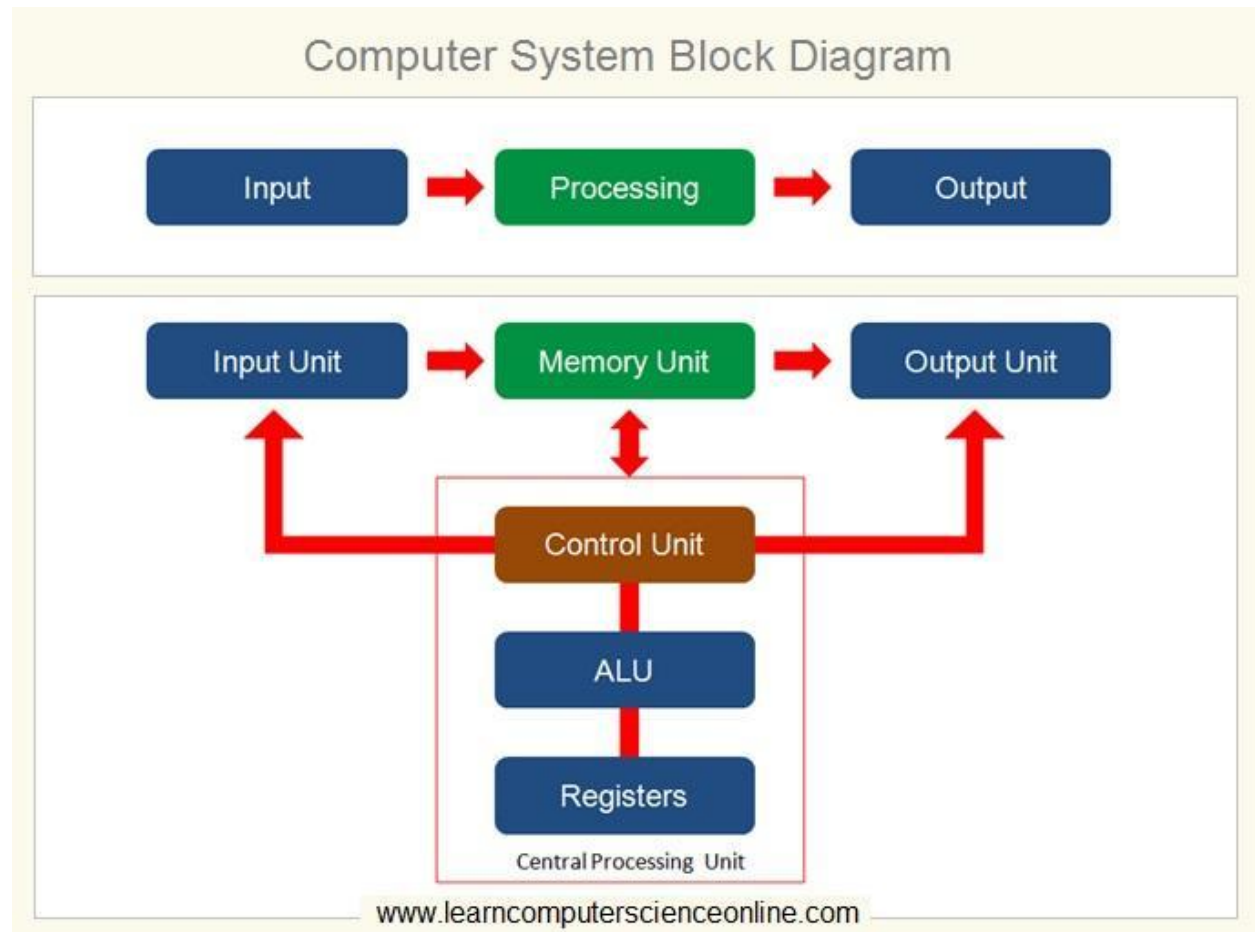
UNIT-1

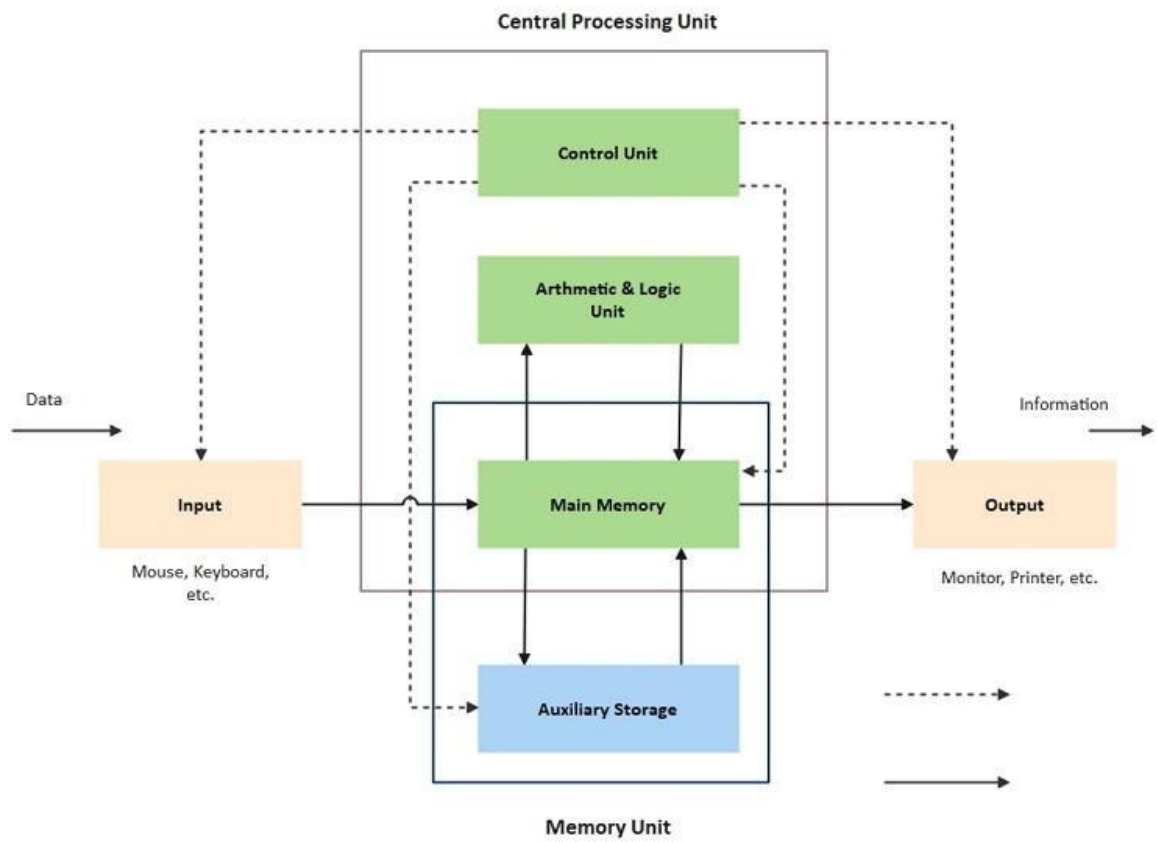
Functional Blocks of a Computer

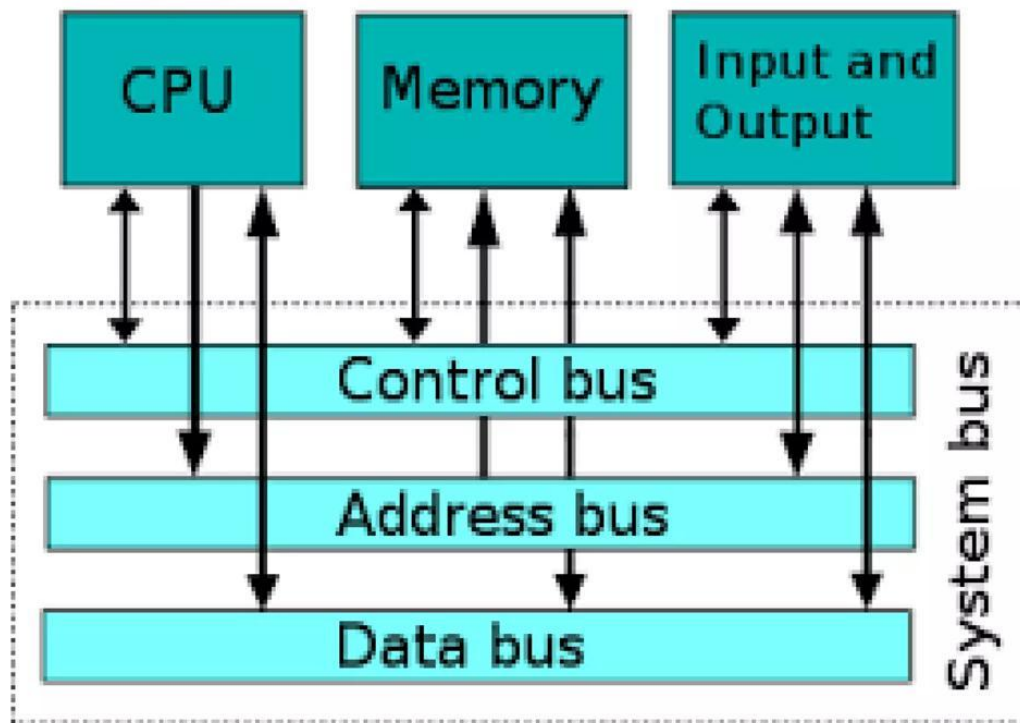
CPU, Memory, Input-Output Subsystem and Control Unit

A computer system works by combining different hardware units together. Every operation performed by a computer, whether opening a browser, playing a game, or compiling a program, happens because these units communicate continuously. The main functional blocks are the CPU, memory, input devices, output devices, and buses.

The CPU or Central Processing Unit is known as the brain of the computer because it performs all calculations and decision-making operations. It executes instructions given by programs. Whenever you run a program like Visual Studio or Chrome, the CPU continuously fetches instructions from memory and processes them.







The CPU mainly contains the ALU, registers, and control unit. The Arithmetic Logic Unit performs arithmetic operations such as addition, subtraction, multiplication, and logical operations like AND, OR, and comparison. Registers are very small and fast memory locations inside the CPU used to temporarily store data. The Control Unit coordinates all operations by sending control signals.

Memory stores instructions and data. RAM temporarily stores currently running programs while ROM permanently stores firmware instructions such as BIOS. Secondary storage like SSD and HDD keeps data permanently.

Input devices such as keyboards, mouse, and scanners send data into the computer. Output devices such as monitors and printers display results. The input-output subsystem handles communication between the CPU and peripheral devices.

For example, when a student types a C program in VS Code and presses Run, the keyboard sends input to memory, the CPU fetches the instructions, processes them, and finally displays the output on the monitor.

Instruction Set Architecture of CPU

Instruction Set Architecture or ISA is the interface between software and hardware. It defines the instructions that the processor can execute. Every CPU follows a particular ISA such as x86, ARM, or RISC-V.

An instruction tells the CPU what operation should be performed. Examples include ADD, SUB, MOV, and JMP. Each instruction has an opcode and operands.

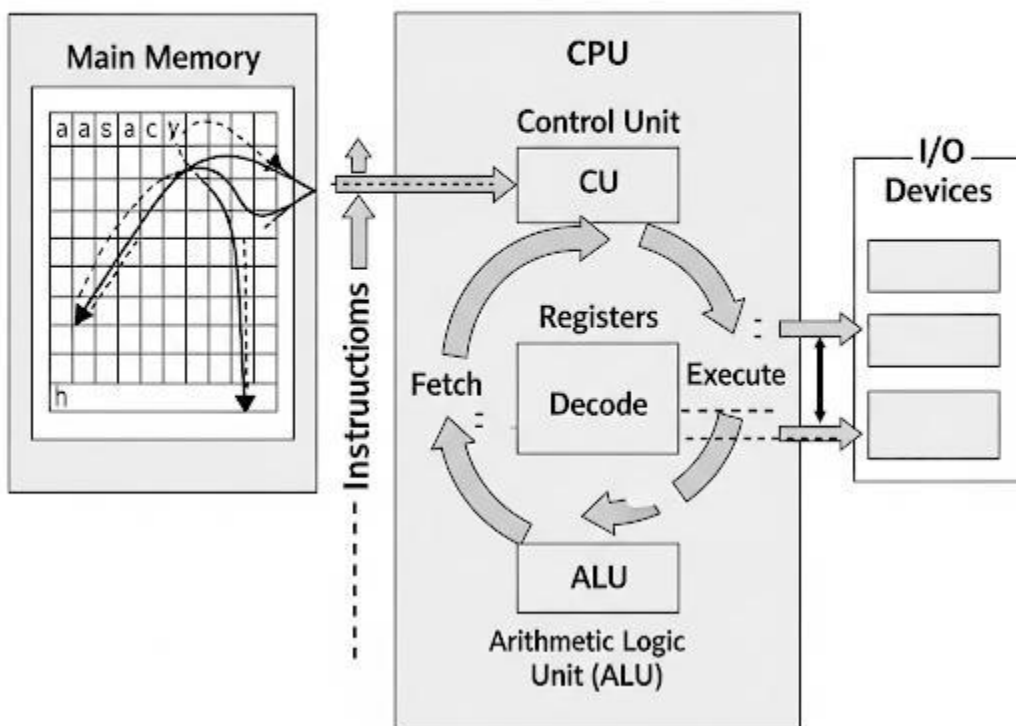
For example:

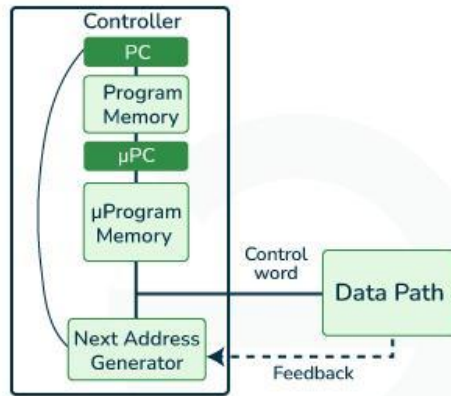
ADD R1, R2

This instruction tells the CPU to add the values stored in registers R1 and R2.

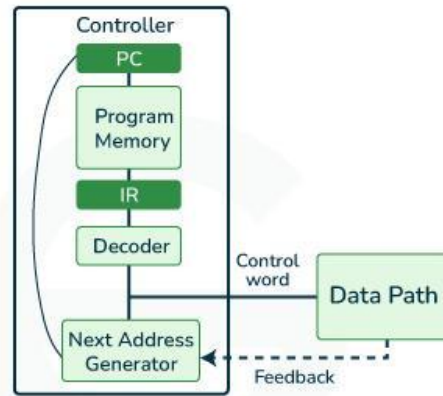
Different architectures provide different instruction formats and capabilities. Complex Instruction Set Computers (CISC) like x86 support many powerful instructions, while Reduced Instruction Set Computers (RISC) focus on simpler and faster instructions.

x86 Architecture and Fetch-Decode-Execute Cycle

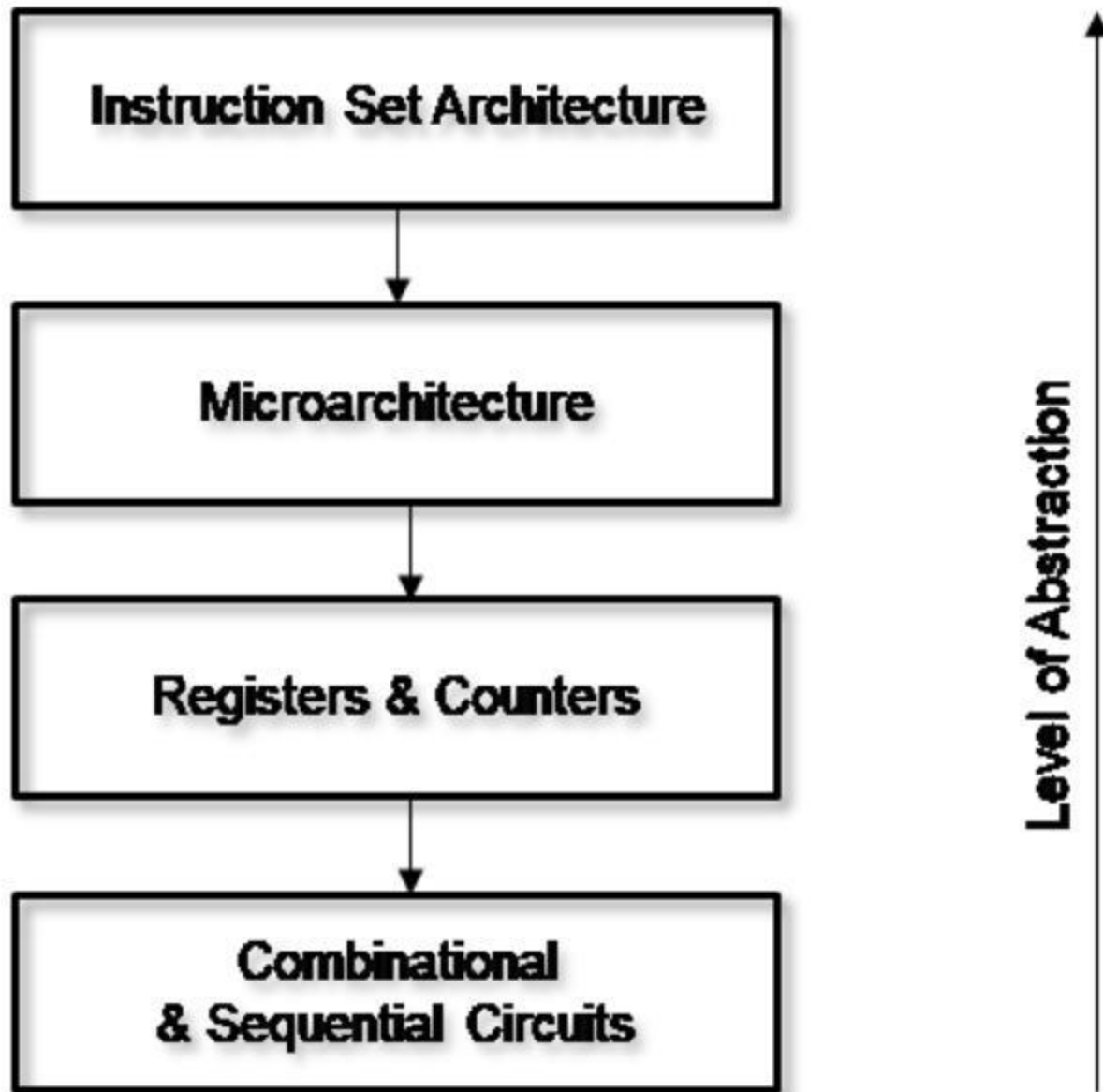




CISC
Complex Instructions Possible
1 Instruction = n μ -Instructions



RISC
Simple Instructions
No μ -programming
 $\text{RISC PM} = N \times \text{CISC PM}$



7

Instruction Set Architecture is important because software developers write programs that are ultimately converted into machine instructions understood by the CPU.

CPU Registers

Registers are tiny storage locations present inside the processor. They are much faster than RAM because they are directly connected to the CPU circuitry.

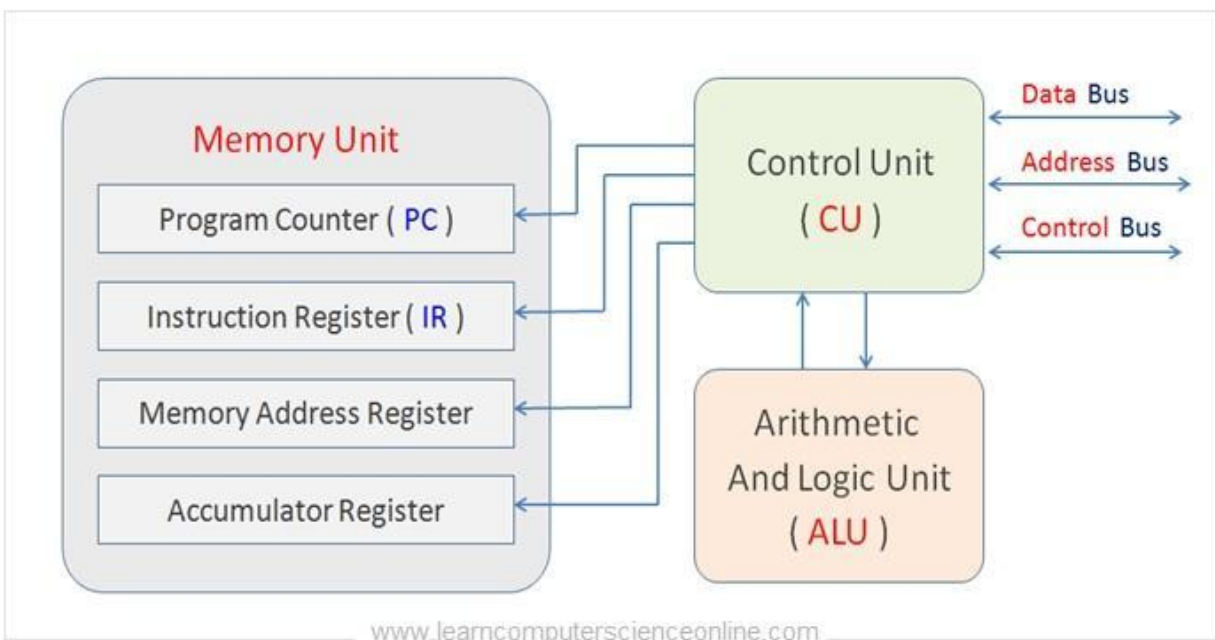
Different registers perform different functions. The Program Counter stores the address of the next instruction to execute. The Instruction Register stores the currently executing instruction. The Accumulator stores intermediate arithmetic results.

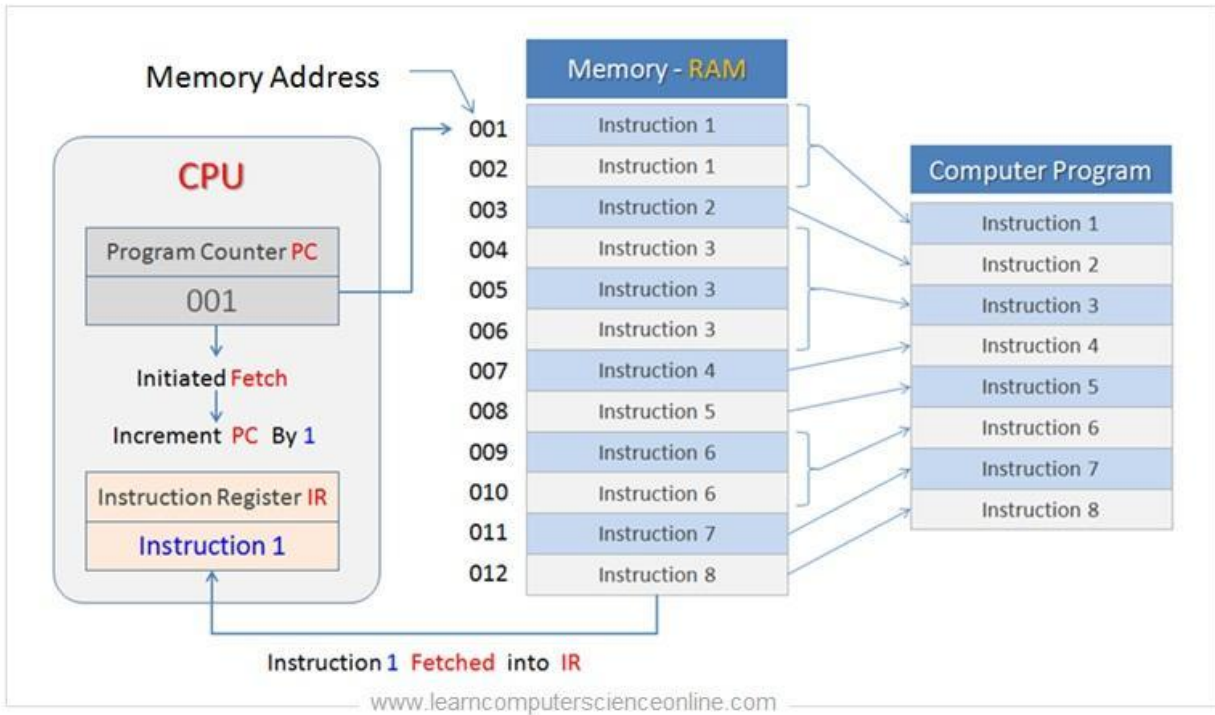
Suppose the CPU executes:

ADD A, B

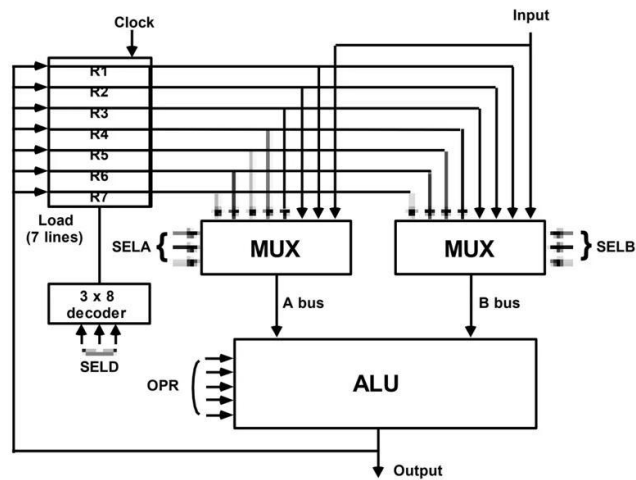
The values of A and B are first loaded into registers. The ALU performs addition, and the result is stored back into another register.

Registers increase processing speed because accessing RAM repeatedly would slow down execution.





GENERAL REGISTER ORGANIZATION



4

6

Instruction Execution Cycle

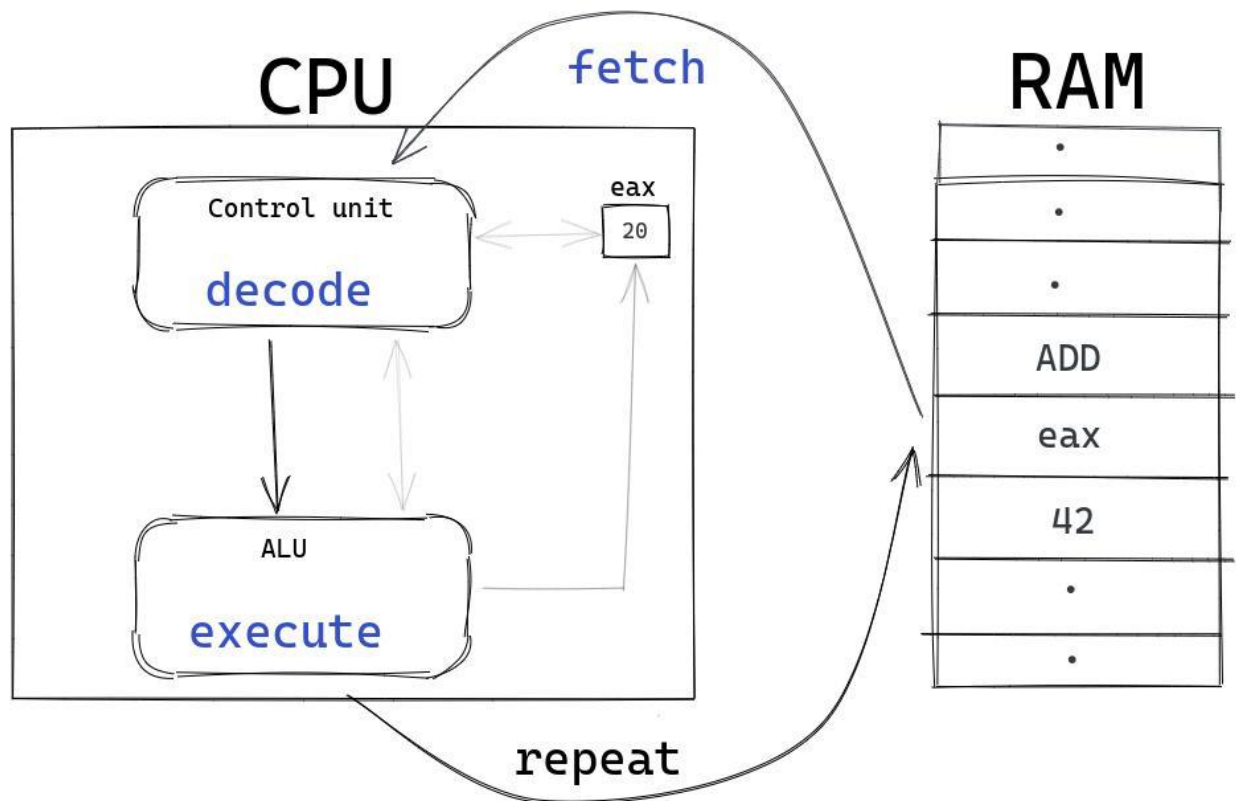
The instruction execution cycle explains how the CPU executes a program step by step. Every instruction passes through three main stages: fetch, decode, and execute.

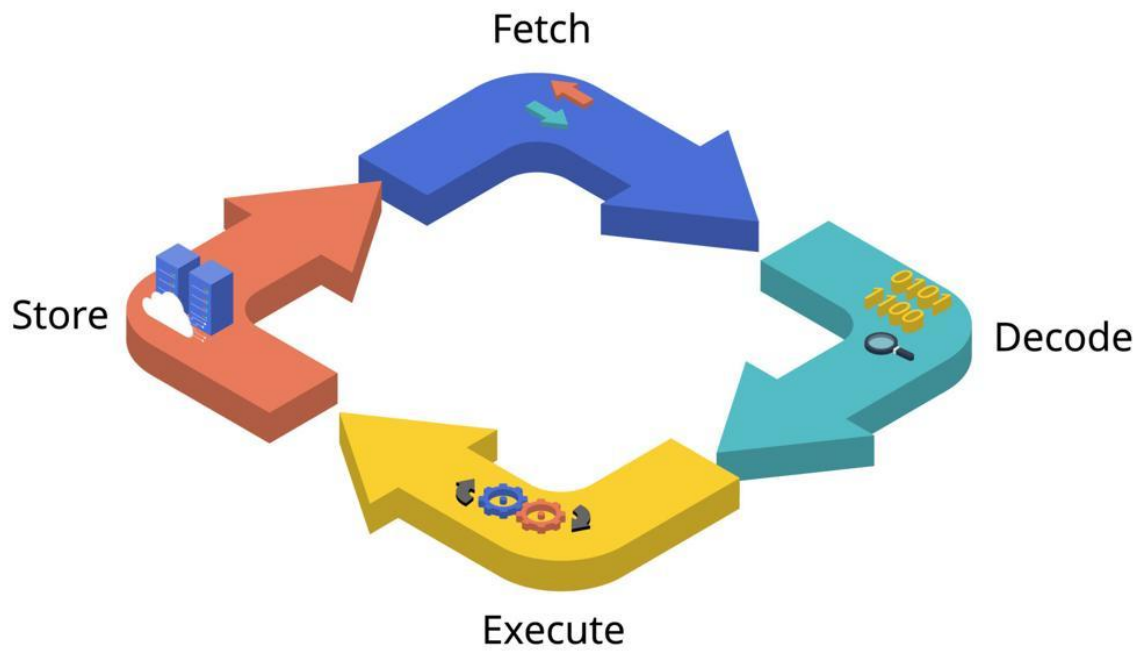
In the fetch stage, the CPU retrieves an instruction from memory using the Program Counter. In the decode stage, the control unit identifies the operation that must be performed. In the execute stage, the operation is carried out.

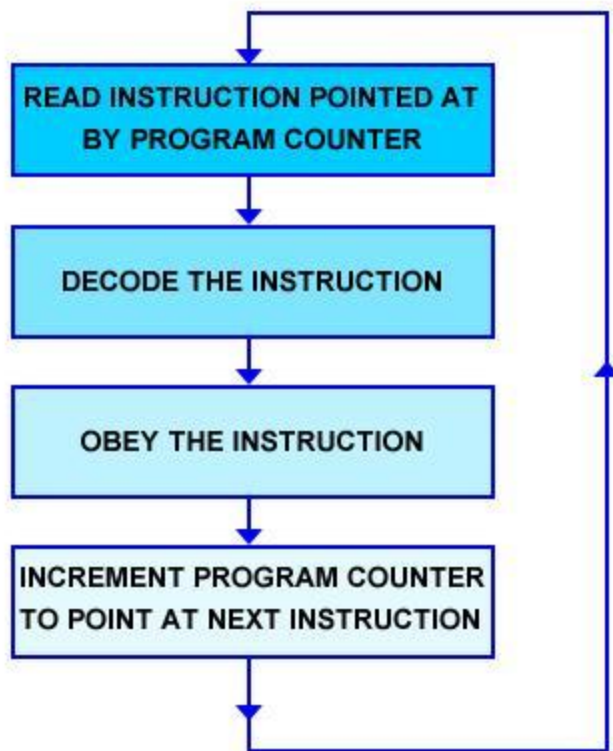
For example, if the instruction is:

ADD R1, R2

The CPU first fetches the instruction from memory, decodes it as an addition operation, and then adds the values stored in R1 and R2.







This cycle repeats millions or billions of times every second in modern processors.

RTL Interpretation of Instructions

RTL or Register Transfer Language is a symbolic language used to describe data movement between registers.

For example:

$$R1 \leftarrow R2$$

means the contents of register R2 are copied into R1.

$$R1 \leftarrow R2$$

If the instruction is:

$$R3 \leftarrow R1 + R2$$

it means the values in R1 and R2 are added and stored in R3.

$$R3 \leftarrow R1 + R2$$

RTL helps engineers understand internal CPU operations clearly.

Addressing Modes

Addressing modes define how operands are accessed by instructions. Different addressing modes provide flexibility and efficiency.

In Immediate Addressing Mode, the actual value is directly present inside the instruction.

Example:

MOV A, 5

Here, the value 5 is directly provided.

In Register Addressing Mode, operands are stored inside registers.

ADD R1, R2

In Direct Addressing Mode, the memory address is specified directly.

MOV A, [5000]

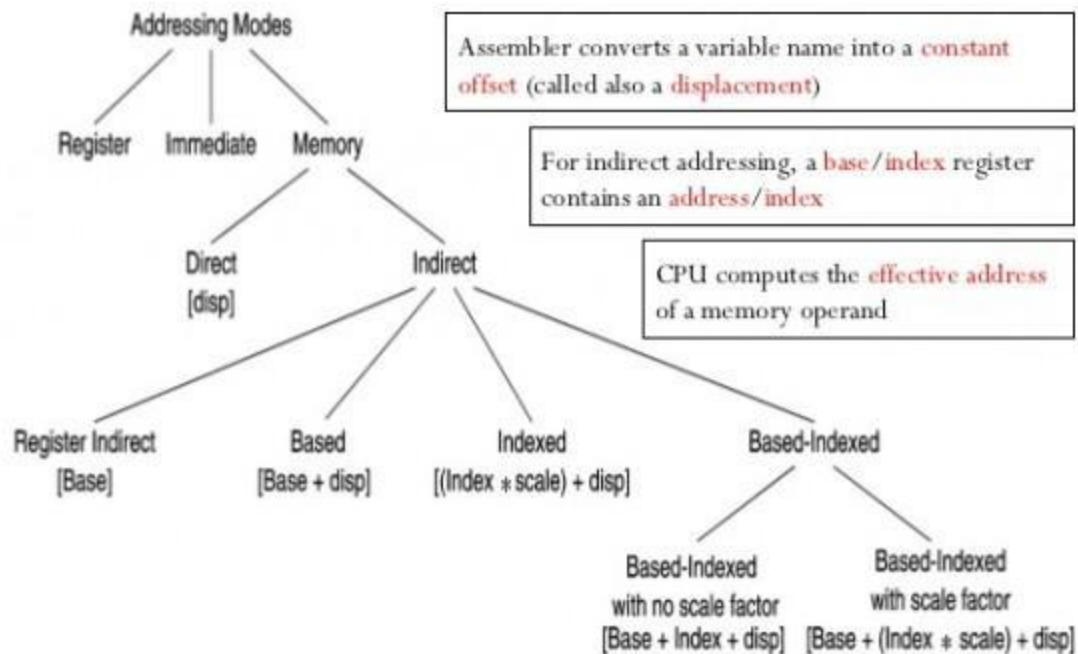
This copies data from memory location 5000 into A.

In Indirect Addressing Mode, the address itself is stored in a register.

Types Of Addressing Mode

- **Implied / Implicit Addressing Mode**
- **Stack Addressing Mode**
- **Immediate Addressing Mode**
- **Direct Addressing Mode**
- **Indirect Addressing Mode**
- **Register Direct Addressing Mode**
- **Register Indirect Addressing Mode**
- **Relative Addressing Mode**
- **Indexed Addressing Mode**
- **Base Register Addressing Mode**
- **Auto-Increment Addressing Mode**
- **Auto-Decrement Addressing Mode**

Summary of Addressing Modes



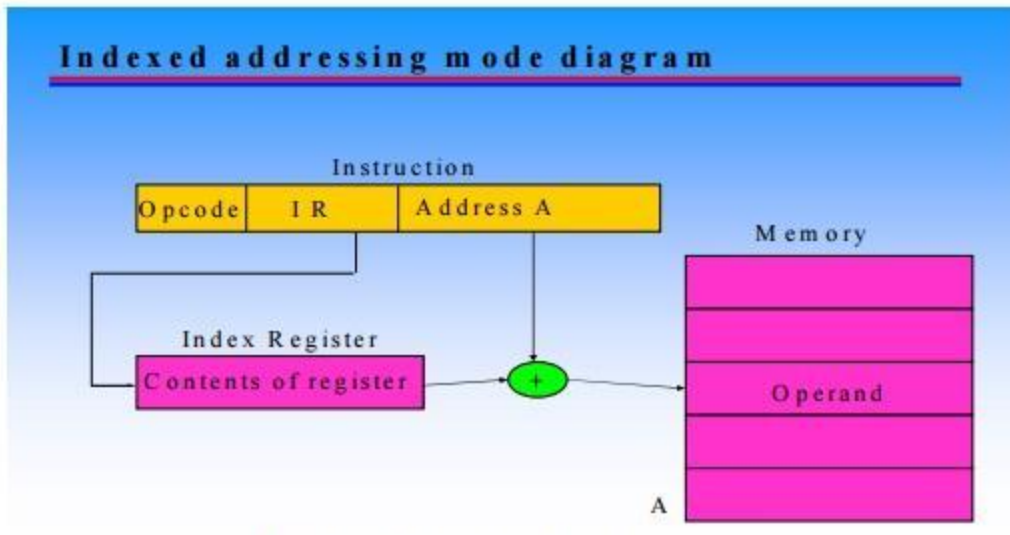


Fig.1.16 Indexed addressing mode

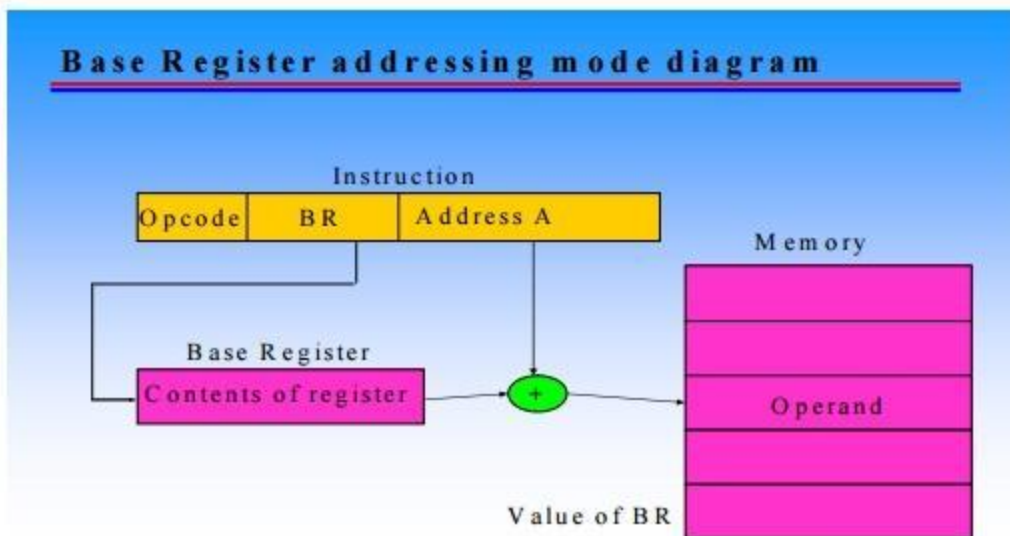


Fig.1.17 Base register addressing mode

Addressing modes improve program flexibility and reduce memory usage.

UNIT-2

Data Representation

Signed Number Representation

Computers understand only binary digits, which are 0 and 1. Positive and negative numbers are represented using special techniques called signed number representations. Earlier systems used sign-magnitude and 1's complement representation, but modern computers mainly use 2's complement representation because arithmetic operations become simpler and faster in hardware.

In 2's complement, the leftmost bit represents the sign. If the bit is 0, the number is positive, and if it is 1, the number is negative. To find the 2's complement of a binary number, first invert all bits and then add 1.

For example, to represent -5 in an 8-bit system:

$$+5 = 00000101$$

Invert bits:

$$11111010$$

Add 1:

$$11111011$$

So:

$$-5 = 11111011$$

$$-5 = 11111011$$

This method is useful because the same adder circuit can perform both addition and subtraction.

Fixed Point and Floating Point Representation

In fixed-point representation, the decimal point position remains fixed. It is simple and fast but cannot represent very large or very small numbers efficiently.

For example:

$$12.34$$

may always store two digits after the decimal point.

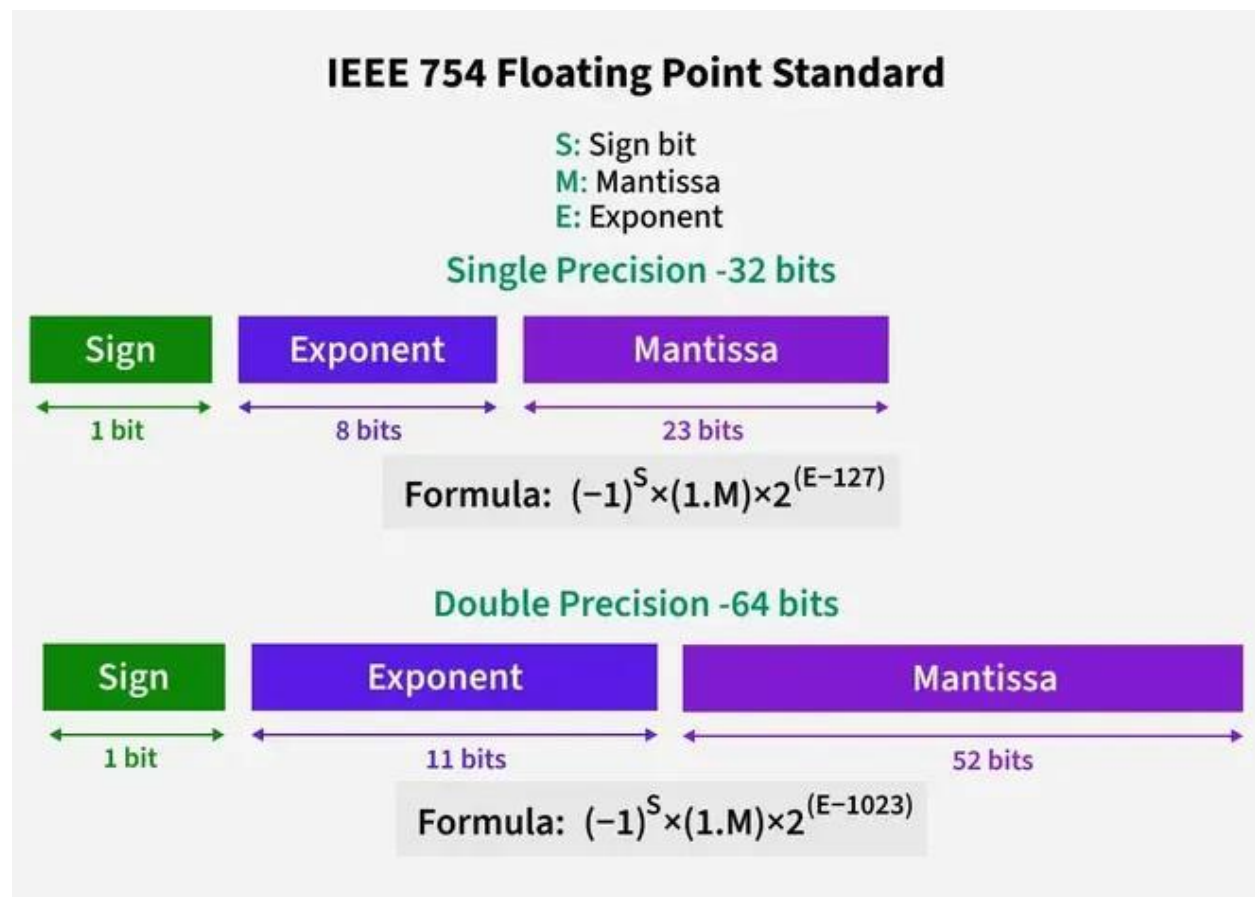
Floating-point representation is more flexible and is used in scientific calculations, graphics, and AI computations. A floating-point number contains three parts: sign bit, exponent, and mantissa.

For example:

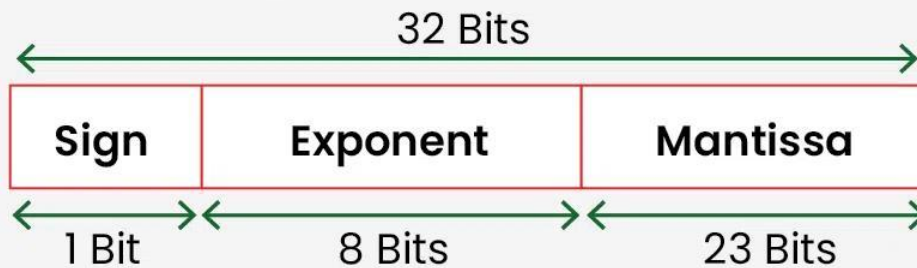
$$6.25 = 110.01_2$$

$$6.25 = 110.01_2$$

Modern computers follow the IEEE 754 standard for floating-point representation.

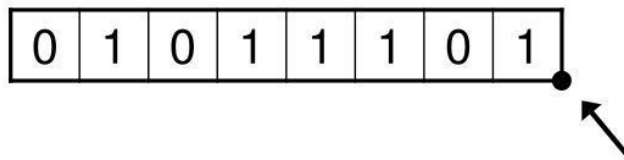


Floating Point Representation



Fixed Point vs. Floating Point

- We've already seen two ways to represent a positive integer in computer hardware: signed and unsigned.
- Both ways were with a **fixed point** representation - the location of the binary point was fixed:



תמר שרוט, נועם חזון

Floating-point numbers allow computers to handle values like:

$$3.14 \times 10^{20}$$

or

$$5.2 \times 10^{-10}$$

efficiently.

Character Representation

Characters are stored using encoding schemes such as ASCII and Unicode. In ASCII, every character is assigned a decimal value.

For example:

$$A = 65$$

and its binary representation is:

$$01000001$$

$$01000001$$

Unicode extends this concept and supports almost all world languages and symbols.

Computer Arithmetic

Binary arithmetic is performed using digital circuits inside the CPU. Addition is carried out using adders.

For example:

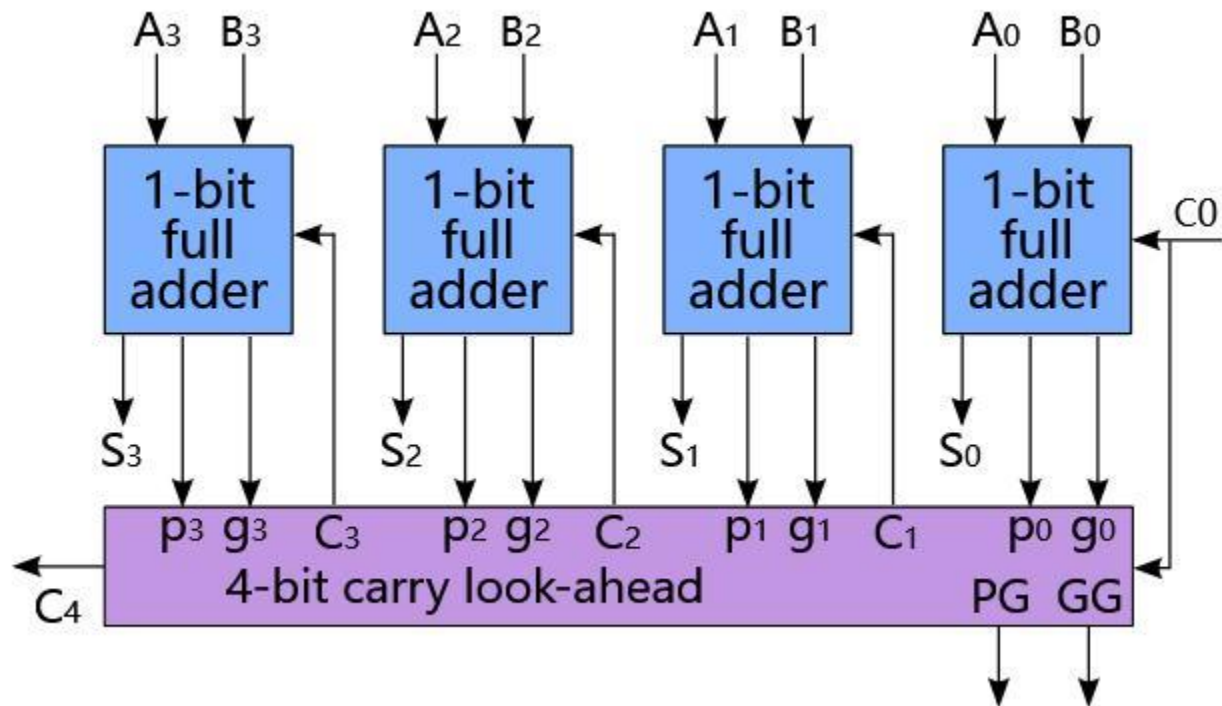
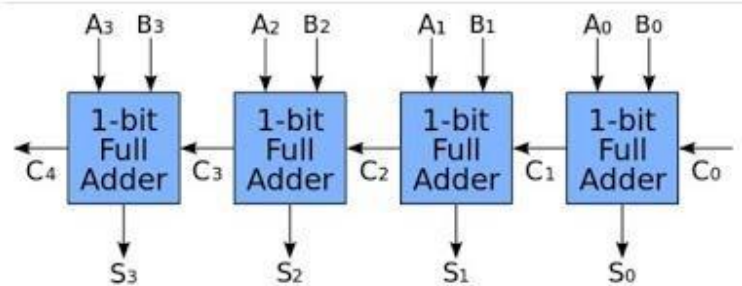
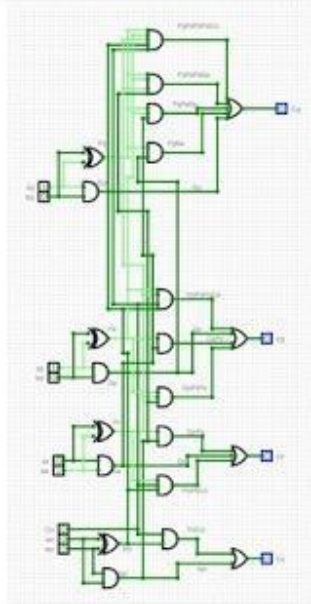
$$1010 + 0101 = 1111$$

$$1010 + 0101 = 1111$$

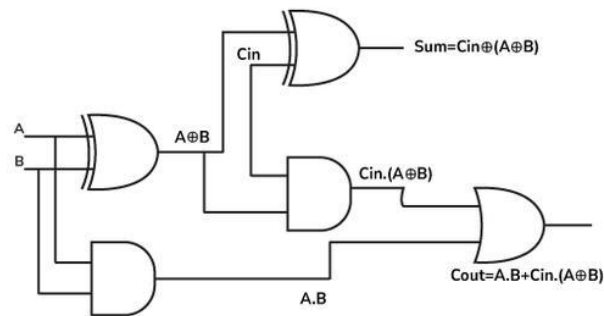
A Ripple Carry Adder performs addition bit by bit, where carry moves from one stage to another. This process is simple but slower because carry propagation takes time.

Carry Look-Ahead Adders improve speed by calculating carry values in advance.

Carry Look Ahead vs Carry Ripple Adder



Full Adder



Inputs			Outputs	
A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4

Subtraction in binary is generally performed using 2's complement representation.

Multiplication Algorithms

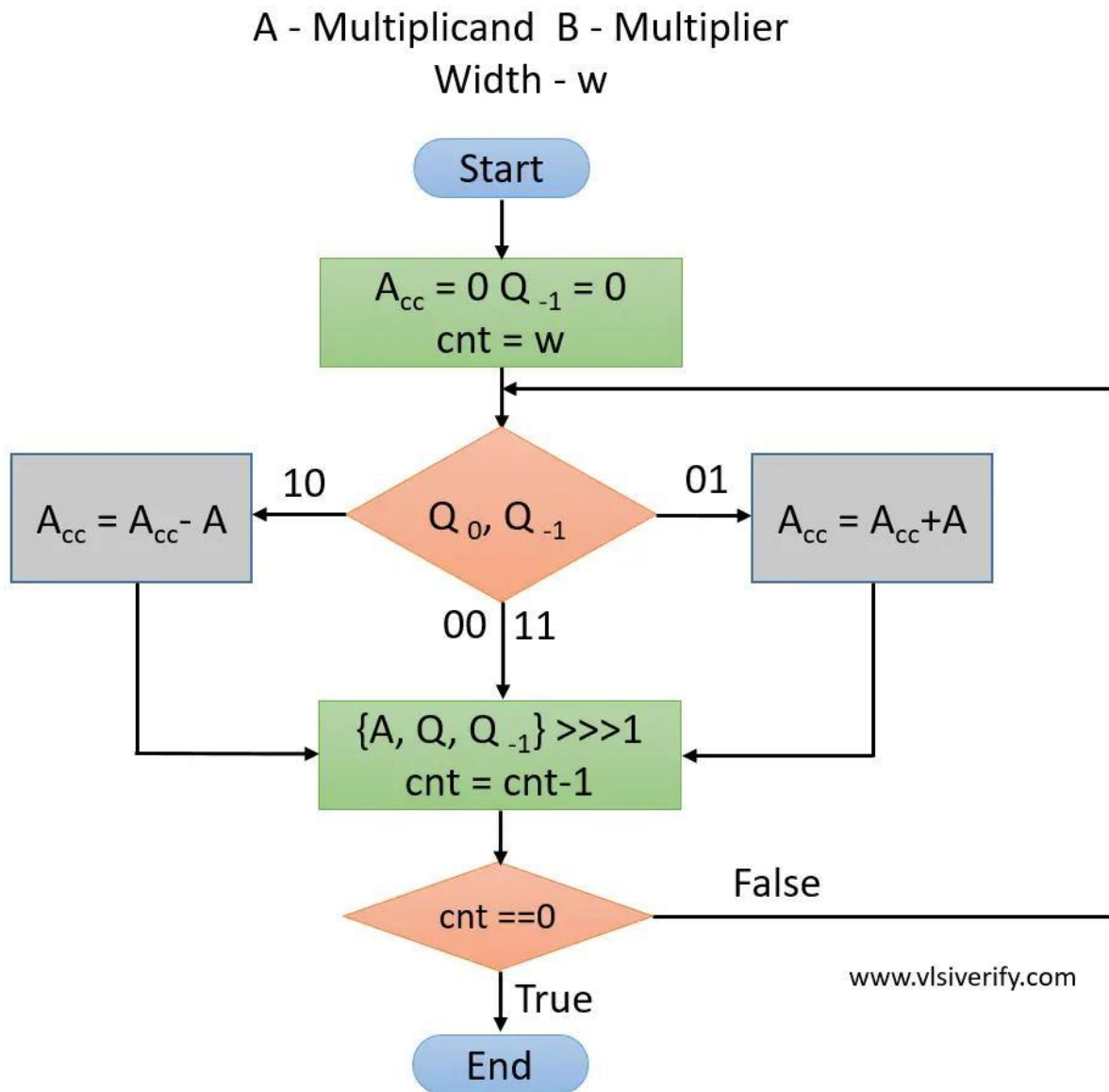
Binary multiplication is similar to decimal multiplication but uses only 0 and 1. Shift-and-add multiplication repeatedly shifts and adds binary numbers.

Booth multiplication is a faster algorithm used for signed binary multiplication. It reduces the number of arithmetic operations required.

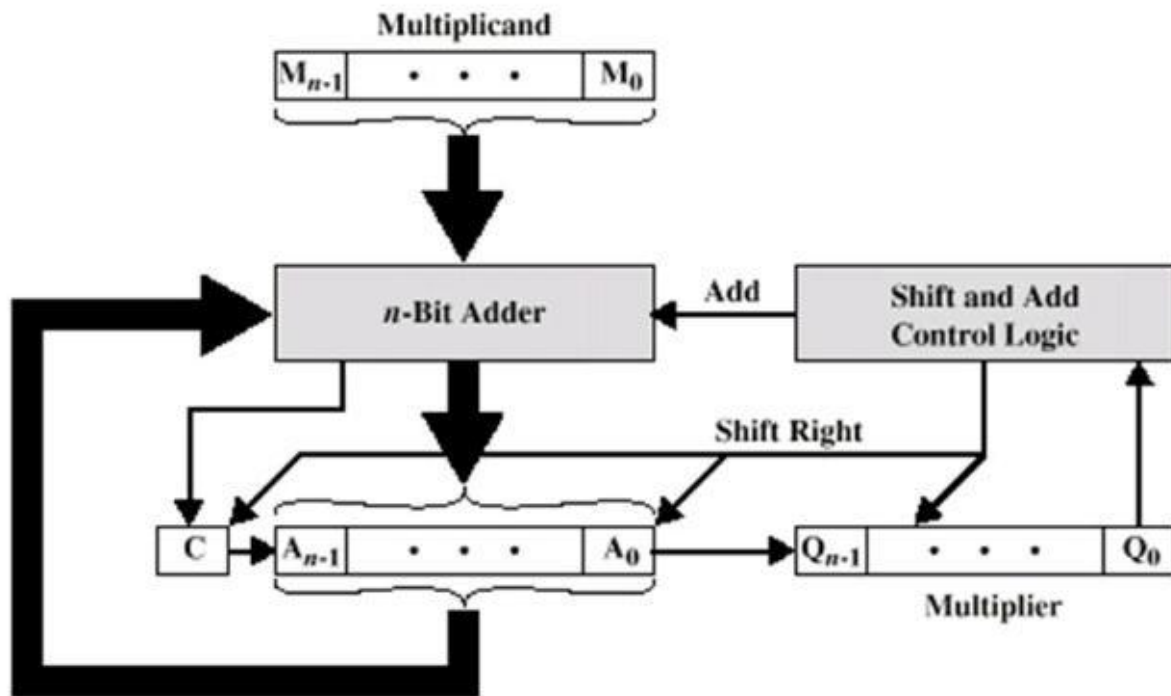
For example:

$$7 \times (-3)$$

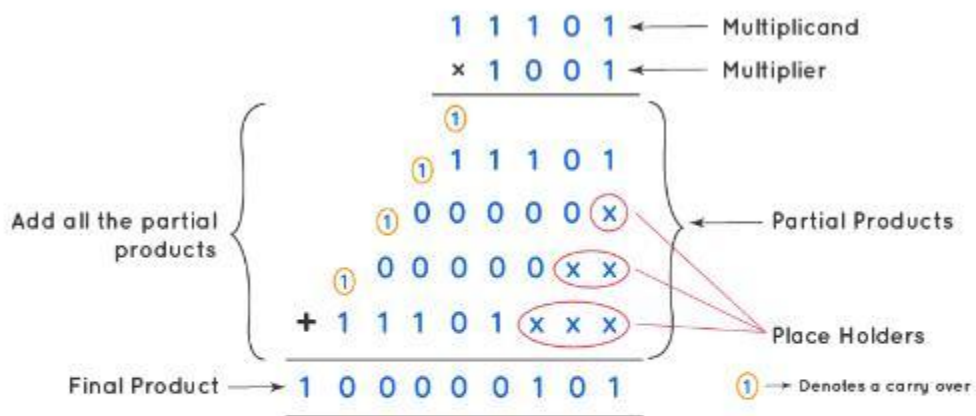
can be efficiently calculated using Booth's algorithm through shifting and addition/subtraction steps.



Booth's Multiplier



(a) Block Diagram



Rules of Binary Multiplication
 $0 \times 0 = 0, 0 \times 1 = 0, 1 \times 0 = 0, 1 \times 1 = 1$
 Rules of Binary Addition
 $0 + 0 = 0, 0 + 1 = 1, 1 + 1 = 10, 1 + 1 + 1 = 11$

Division Algorithms

Division in computers is performed using restoring and non-restoring algorithms.

In restoring division, if subtraction gives a negative result, the original value is restored before continuing. This method is simple but slower.

In non-restoring division, restoration is avoided, making the process faster.

For example:

$$13 \div 3 = 4$$

with remainder:

$$1$$

$$13 \div 3 = 4$$

These algorithms are implemented using hardware registers and subtractor circuits.

Floating Point Arithmetic

Floating-point arithmetic is more complex than integer arithmetic because the exponent and mantissa must be handled separately.

During addition, exponents are first aligned.

For example:

$$1.5 \times 10^2 + 2.5 \times 10^1$$

The smaller exponent must first be converted before addition is performed.

$$1.5 \times 10^2 + 2.5 \times 10^1$$

After the operation, normalization is performed to keep the result in standard scientific notation.